

Aritmética del Computador

El Sistema de Punto Flotante y sus Consecuencias Ocultas

Dr. Hermes Pantoja Carhuavilca

Empecemos con una simple pregunta: ¿Cuánto es $0.1 + 0.2$?

```
>>> 0.1 + 0.2  
0.300000000000000000000000000004
```

Confiamos en las computadoras para cálculos de alta precisión. Sin embargo, fallan en la aritmética más básica.

Esto no es un error aislado. Es una característica fundamental de su diseño.

¿Por qué ocurre esto y qué implicaciones tiene para la ingeniería?

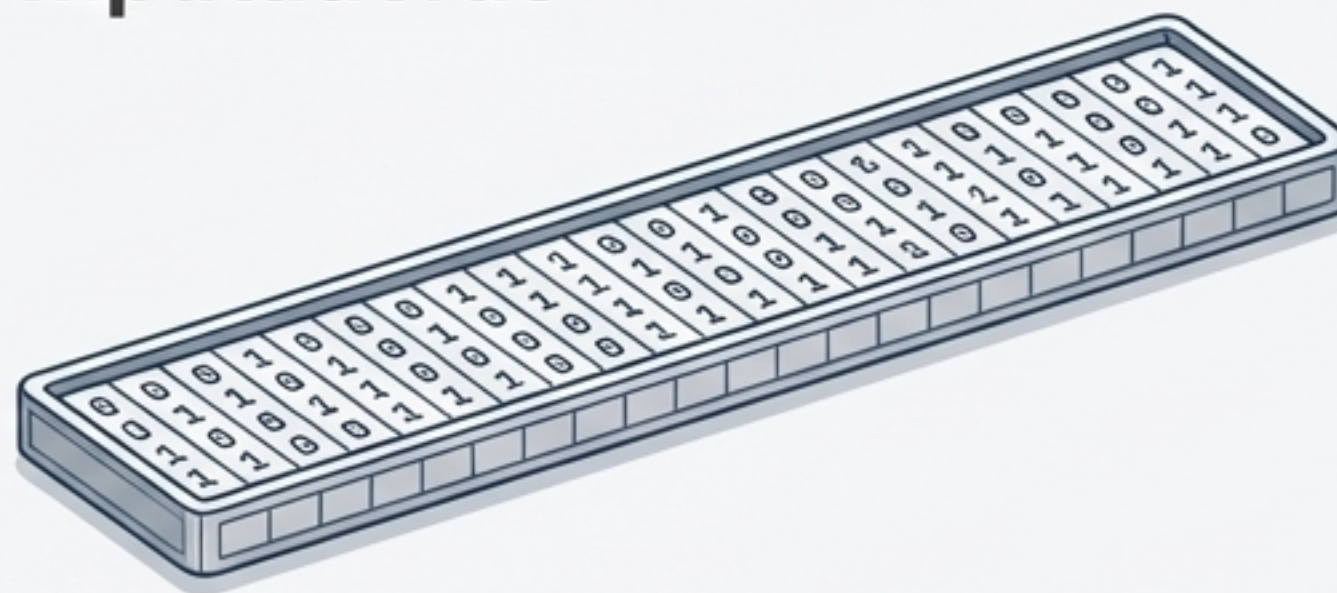
El Origen del Problema: Mundos Diferentes, Lenguajes Diferentes

Humanos



Sistema decimal (Base 10). Precisión teóricamente infinita. Números como π o $1/3$ se manejan conceptualmente sin pérdida.

Computadoras

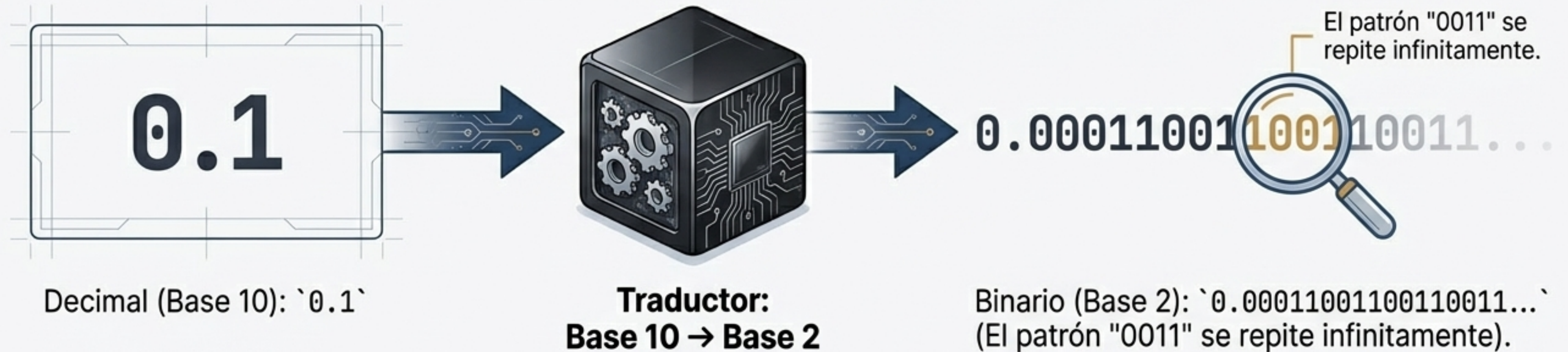


Sistema binario (Base 2). Memoria y precisión finitas. Todo número debe caber en un espacio predefinido (ej. 32 o 64 bits).

Para representar un rango inmenso de números en un espacio finito, las computadoras no usan los números decimales que vemos. Usan una aproximación ingeniosa pero imperfecta: la **notación científica binaria**, o **Punto Flotante**.

La "Traducción" Imposible: Por qué 0.1 no es 0.1 para una computadora.

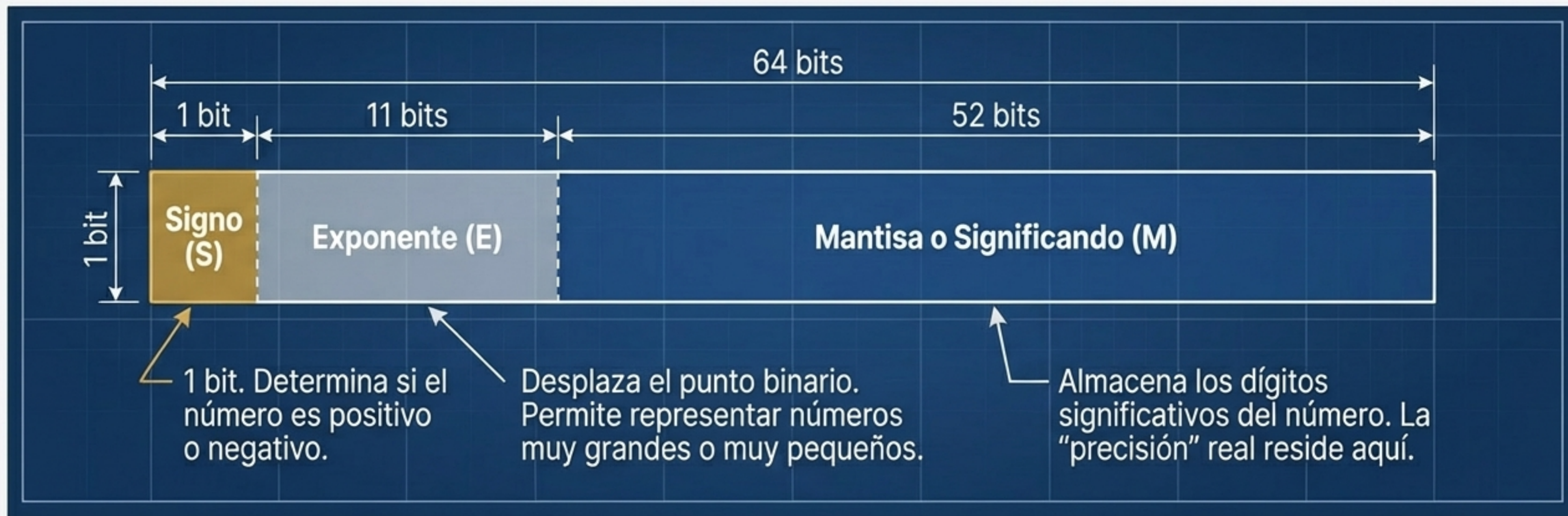
En base 10, fracciones como $\frac{1}{3}$ se vuelven **decimales infinitos (0.333...)**. En base 2, ocurre lo mismo con fracciones simples en base 10.



La computadora no puede almacenar una secuencia infinita de bits. **Debe cortar (truncar) la representación.** Este es el primer e inevitable error. El número que la máquina almacena no es 0.1, sino su aproximación binaria más cercana.

La Arquitectura de los Números: El Estándar IEEE 754.

Para garantizar la consistencia entre diferentes máquinas, la industria adoptó el estándar IEEE 754. Define la estructura precisa para almacenar números de punto flotante.

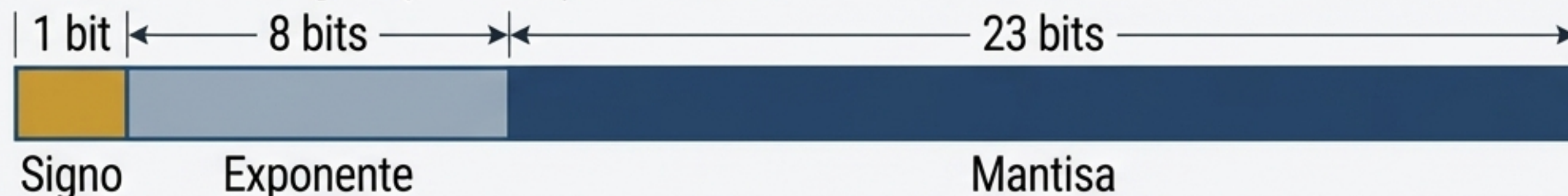


Fórmula General

$$\text{Número} = (-1)^{^S} * (1.\text{Mantisa}) * 2^{(\text{Exponente} - \text{sesgo})}$$

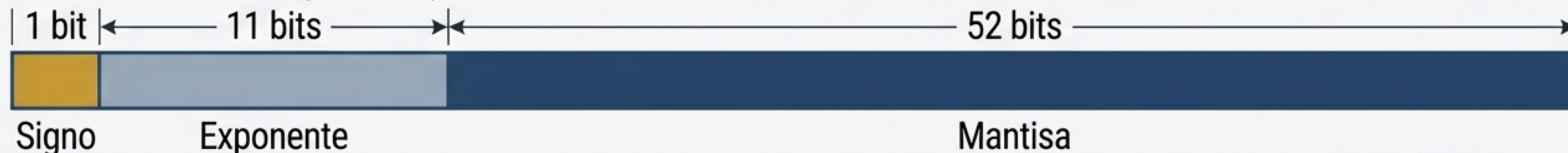
Los Dos Formatos Clave: Precisión Simple vs. Doble.

Precisión Simple (32 bits) - `float`



Uso: Común en gráficos por computadora, donde la velocidad es más crítica que la máxima precisión.

Precisión Doble (64 bits) - `double`



El estándar por defecto en la mayoría de los lenguajes científicos y de ingeniería (Python, MATLAB, R, C).

Concepto Clave: Normalización

Para maximizar la precisión, los números se almacenan de forma que el primer dígito de la mantisa sea siempre 1 (ej. `1.xxxxx...`). Este '1' se vuelve **implícito** (no se almacena), regalando un bit extra de precisión.

1.xxxxx...
---- Bit Implícito

El Límite de la Percepción: Épsilon de Máquina (ϵ).

Definición Conceptual

Debido a la precisión finita, existe un “salto” entre un número representable y el siguiente. El **Épsilon de Máquina** es la distancia entre **1.0** y el siguiente número más grande representable.

Definición Formal

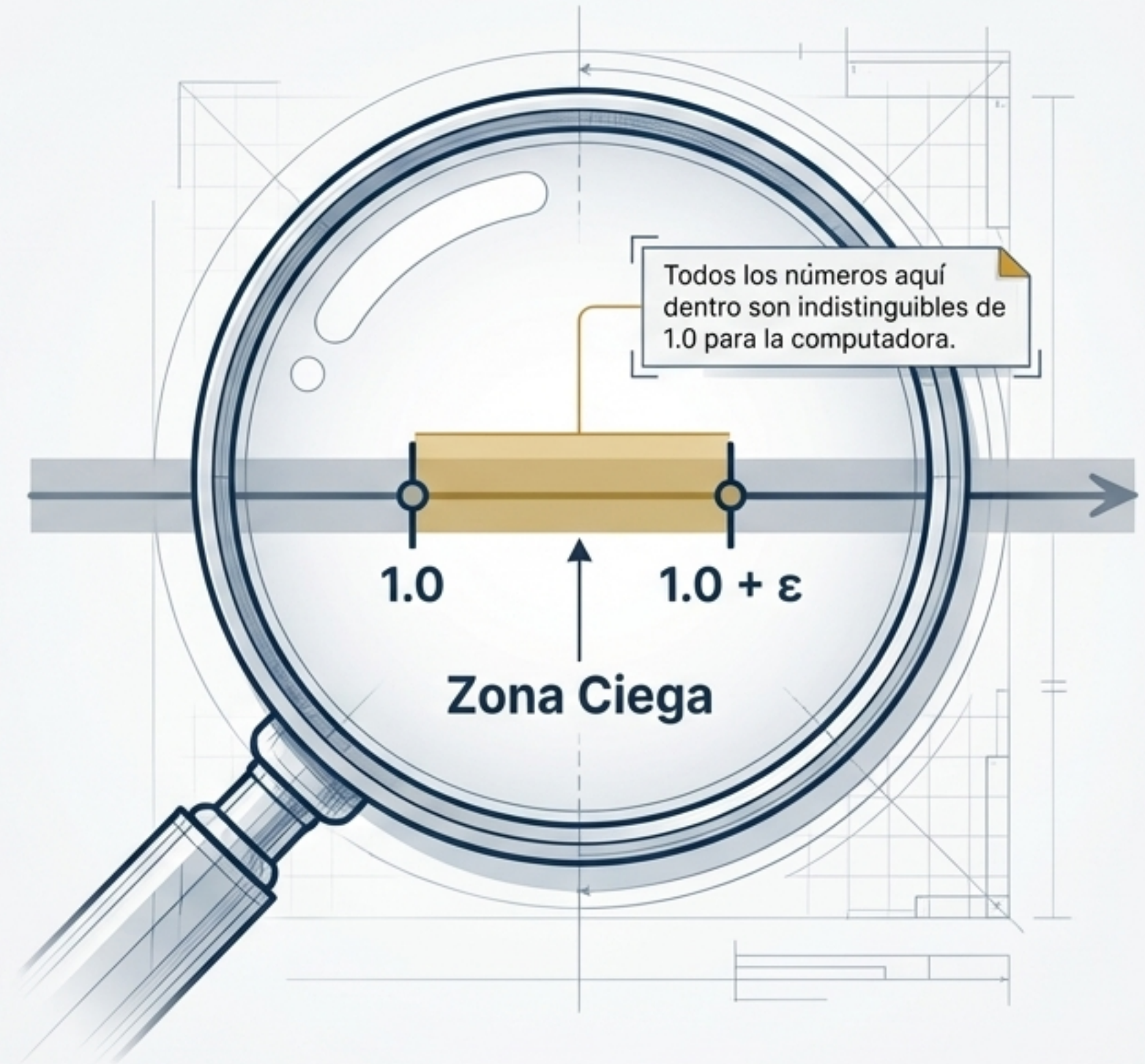
ϵ es el número más pequeño tal que $1.0 + \epsilon$ es diferente de 1.0 en la computadora.

Valores Típicos

Precisión Simple (32-bit): $\epsilon \approx 1.19 \times 10^{-7}$

Precisión Doble (64-bit): $\epsilon \approx 2.22 \times 10^{-16}$

****Cualquier cálculo cuyo resultado sea más pequeño que Épsilon (relativo a la magnitud) se perderá. Es el ‘ruido de fondo’ del cómputo numérico.**



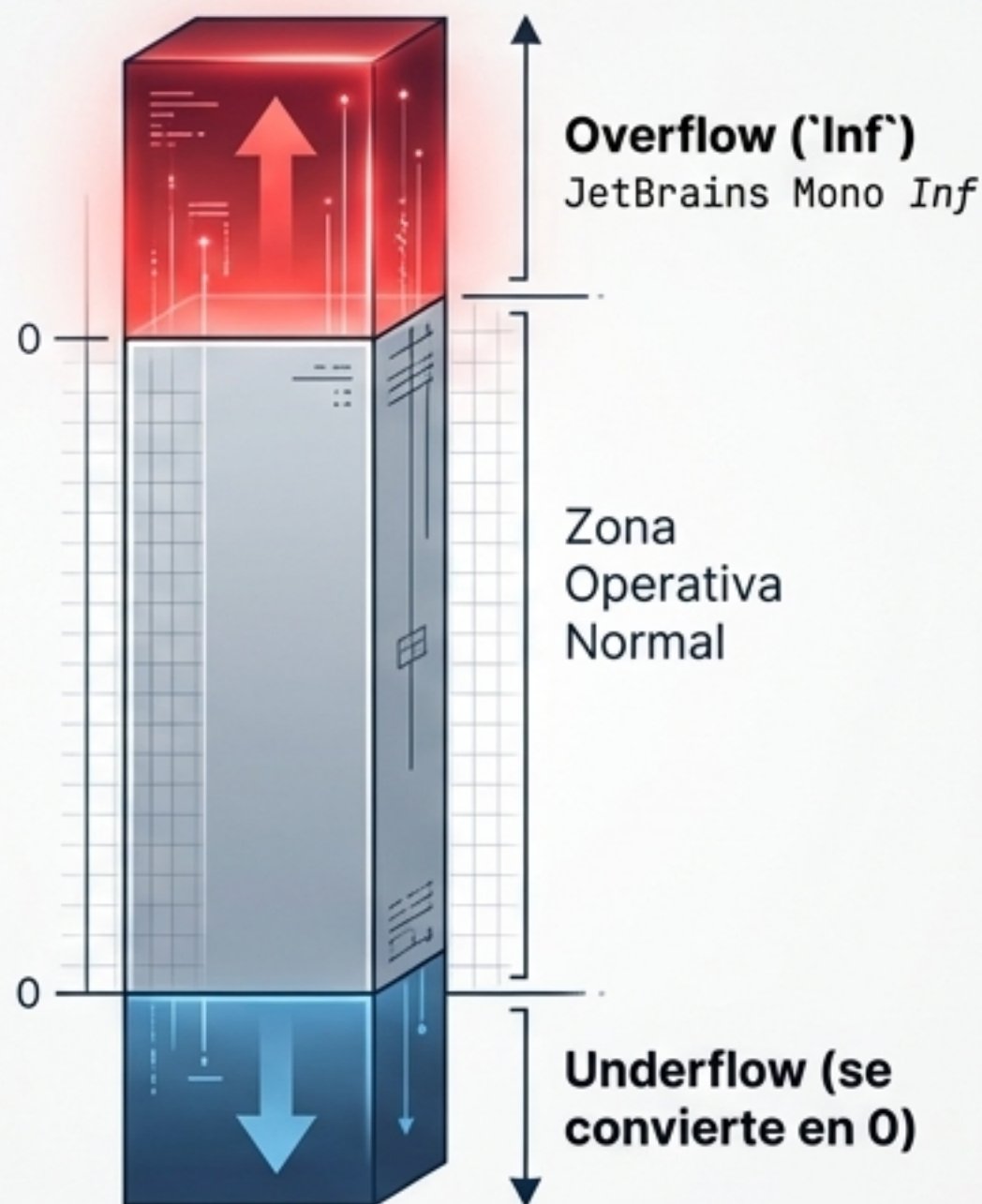
Los Límites del Universo Numérico: Overflow y Underflow.

Overflow (Desbordamiento)

Qué es: Ocurre cuando el resultado de una operación es más grande que el número máximo representable en el formato.

Resultado: El sistema devuelve un valor especial: ``Inf`` (Infinito).

Peligro: ``Inf`` puede propagarse a través de los cálculos, invalidando resultados finales.



Underflow

Qué es: Ocurre cuando el resultado de una operación es más pequeño (más cercano a cero) que el número mínimo positivo representable.

Resultado: El número es 'redondeado' a cero.

Peligro: La pérdida de información puede ser sutil pero catastrófica en algoritmos que dependen de valores muy pequeños (ej. probabilidades, cálculos de gradientes).

El Error Más Peligroso: Cancelación por Sustracción

El Problema

¿Qué sucede cuando restamos dos números de punto flotante muy cercanos y casi iguales?

Explicación

Los dígitos principales, que contenían la información precisa, se han cancelado. Los dígitos que quedan son esencialmente “ruido” magnificado, provenientes de los errores de redondeo previos en `x` e `y`.

Hemos perdido casi toda la precisión útil.

“Esto es fatal en algoritmos de control de vuelo, balística, o cualquier sistema que dependa de la diferenciación numérica de señales ruidosas.”

$$x = 0.1234567891234567$$

$$y = 0.1234567891234512$$

$$x - y = 0.0000000000000055$$

Renormalización

$$\text{Resultado} = 5.500000000000000... \times 10^{-15}$$

Un Ejemplo en el Código: La Raíz de una Ecuación Cuadrática.

Contexto: Resolver $ax^2 + bx + c = 0$ usando la fórmula cuadrática: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

El Peligro: Si b^2 es mucho más grande que $4ac$, entonces $\sqrt{b^2 - 4ac} \approx b$.

El Fallo: La expresión $-b + \sqrt{b^2 - 4ac}$ se convierte en una resta de dos números casi iguales. Esto provoca una cancelación catastrófica.

Ingenuo y Peligroso ❌

```
// Si b es grande y positivo, x1 sufre de cancelación
x1 = (-b + sqrt(b*b - 4*a*c)) / (2*a);
x2 = (-b - sqrt(b*b - 4*a*c)) / (2*a);
```

Numéricamente Estable ✅

```
// Evita la cancelación usando una formulación alternativa
if (b > 0) {
    x1 = (-b - sqrt(b*b - 4*a*c)) / (2*a);
} else {
    x1 = (-b + sqrt(b*b - 4*a*c)) / (2*a);
}
x2 = c / (a * x1); // Usa la relación de Vieta (x1*x2 = c/a)
```

Lección: El conocimiento de la aritmética del computador nos permite elegir un algoritmo que es matemáticamente equivalente pero numéricamente superior.

El Efecto Dominó: Acumulación de Errores

Un solo error de redondeo es minúsculo. Pero, ¿qué pasa cuando se realizan miles de millones de operaciones?

Case de Estudio: Simulaciones Climáticas

Modelos que simulan el clima durante décadas. Cada paso de tiempo introduce un pequeño error de redondeo. Después de millones de pasos, el error acumulado puede ser mayor que la señal que se está midiendo.

Error de redondeo (ϵ)

Resultado del Modelo Inválido

Case de Estudio: Sumatorias de Larga Duración

Calcular la suma de una serie con millones de términos. Si se suman números pequeños a un total grande, su contribución puede perderse.

Solución: Sumar la serie desde los términos más pequeños hacia los más grandes para minimizar la pérdida de precisión.

Caso de Estudio: El Desastre del Misil Patriot (Guerra del Golfo, 1991).

Contexto

El sistema de defensa antimisiles Patriot necesitaba calcular la trayectoria de misiles Scud entrantes basándose en el tiempo.

El Fallo Técnico

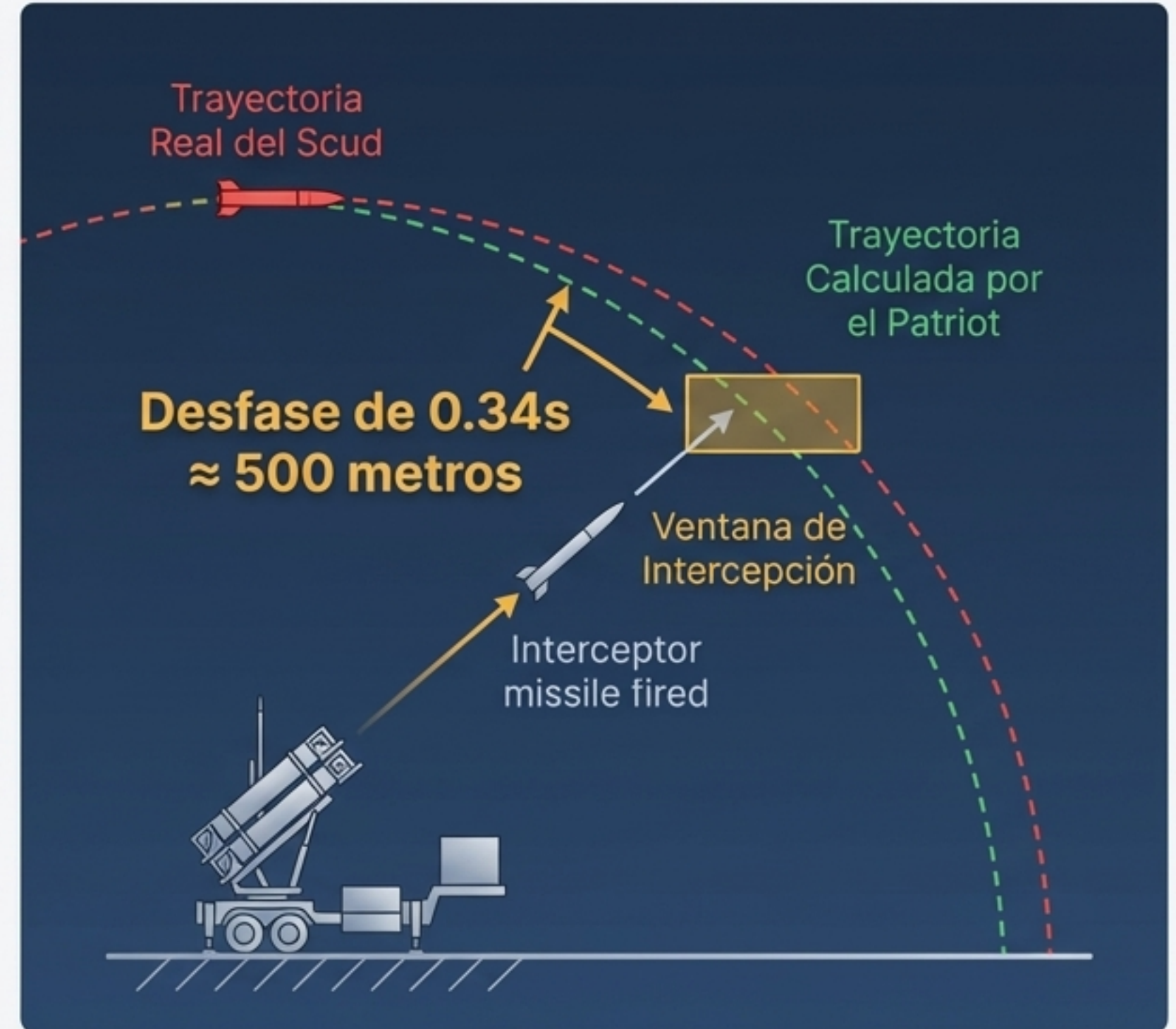
El reloj interno medía el tiempo en décimas de segundo. El número $1/10$ tiene una representación binaria infinita. El sistema usaba registros de 24 bits, y la representación binaria de $1/10$ fue truncada, introduciendo un error minúsculo.

La Acumulación del Error

Este pequeño error, al multiplicarse por el gran número de décimas de segundo en 100 horas de operación, resultó en un desfase de **0.34 segundos**.

La Consecuencia

El misil no fue interceptado. Impactó un cuartel del ejército de EE.UU., causando **28 muertes**.





La Lección del Patriot: La Precisión no es un Detalle Académico.

Análisis del Fallo

- **Error:** No era un bug de software en el sentido tradicional. El código hacía lo que se le pedía.
- **Causa Raíz:** Una falta de comprensión fundamental sobre cómo las computadoras manejan la aritmética de punto flotante.
- **El Costo:** Un error de truncamiento de menos de 0.0000001 segundos se acumuló hasta convertirse en una catástrofe.

A	B	C	D	E	F
	P0	K	k	t	P(t)
	100	1000	0.1	0	100
mento:	h = 5			5	142,1893
				10	190,2569
				15	269,4875
				20	354,9789
				25	450,8531
				30	550,5209
				35	646,291
				40	731,6839
				45	800,6039

Cita Clave: Este es el caso de estudio definitivo sobre la importancia crítica de la aritmética del computador. Demuestra que la elección de la precisión y el manejo de los errores de redondeo son una responsabilidad central del ingeniero, no un problema secundario.

Resumen de Competencias Adquiridas

Al finalizar esta sesión, usted puede:

- 1 **Explicar** por qué las computadoras usan punto flotante y cómo el estándar IEEE 754 estructura los números reales en un formato binario finito.
- 2 **Identificar** las limitaciones inherentes del sistema: errores de redondeo, overflow, underflow y cancelación por sustracción.
- 3 **Calcular y entender** el significado del Épsilon de Máquina (ϵ) como el límite de la resolución del sistema.
- 4 **Predecir** fallos numéricos potenciales en algoritmos comunes y reconocer la importancia de la estabilidad numérica.
- 5 **Distintuir** entre la precisión adecuada (simple vs. doble) para una tarea de ingeniería, priorizando la seguridad y la robustez.

La Responsabilidad del Ingeniero: Más Allá de Escribir Código

El Veredicto: La aritmética de punto flotante es una herramienta poderosa pero fundamentalmente imperfecta.



- No basta con saber programar; es imperativo entender que los números en la computadora son “**discretos**” y son **aproximaciones**.
- El **conocimiento de la aritmética subyacente** no es opcional. Es lo que separa un **diseño robusto y seguro** de un fallo catastrófico.
- La **estabilidad numérica** debe ser siempre una **prioridad** sobre la simple velocidad de cálculo.

El Número que Ves no es el Número que Es.

Punta del Iceberg (visible):

$0.1 + 0.2 = 0.300000000000000004$.

Un simple 'glitch'.

Masa Sumergida (invisible):

El sistema de punto flotante, errores de redondeo, cancelación catastrófica, el estándar IEEE 754 y la responsabilidad del ingeniero sobre la vida y la seguridad.

La próxima vez que vea un resultado numérico inesperado, no lo descarte como un error. Véalo como un recordatorio de la compleja y frágil maquinaria que sustenta nuestro mundo digital. Comprenderla es su deber.